ezdyn: Sufficient-Statistics Analysis and Dashboarding of Dynamic Economic Models in R

September 2026*

Abstract

ezdyn is an R package that aims to make completing policy analysis with (suites of) dynamic economic models frictionless. ezdyn can complete analytical tasks (e.g. impulse response analysis, historical shock decompositions, policy counterfactuals, and optimal policy exercises) across suites of models in a single line of code, or via a code-free interactive dashboard in your web browser.
ezdyn implements recently developed techniques from the "sufficient-statistics" literature on policy counterfactuals and optimal policy.
This document details both how the package can be used and the underlying sufficient-statistics and partial-anticipation theory.

Keywords: DSGE models; impulse responses; forward guidance; sufficient statistics; optimal monetary policy; R software.

---

*Most of ezdyn, the ezDynare toolkit, and the ezdyn dashboard were written prior to the significant advances in generative AI (GAI) in 2026. GAI tools were used since to augment the product with additional features and to add unit tests.

Contents

1 Introduction **3**

2 The `ezdyn` dashboard **4**

3 `ezdyn` on the command line **7**

4 Theoretical background **10**
4.1 Sufficient statistics for policy counterfactuals and optimal policy 10

Appendices **10**

A Getting started guide — Installation and setup **11**
A.1 Installing the package 11
A.2 Setting up a model file 11
A.3 Setting up a model metadata workbook 12
A.4 Loading in your model 13
A.5 Setting up an ezdyn dashboard 13

B Algorithm for solving for optimal policy **14**

1 Introduction

Researchers and policy analysts routinely work with multiple dynamic models (and variants of models) side by side when completing analytical tasks. This is currently harder than it should be. Different models are implemented in different software, use different variable names and units. Even a simple exercise, such as plotting the response of inflation to a given shock in two models on the same axes, typically requires bespoke, model-specific code to be written and audited each time. Dashboards are often developed separately for each model, adding to a significant maintenance and user-friendliness burden.

At the same time, a growing "sufficient-statistics" literature has shown that many policy questions can be answered using only a handful of objects: a baseline forecast, a set of preferences over outcomes, and a set of impulse responses describing the causal effect of policy on those outcomes (e.g. Barnichon and Mesters, 2023; McKay and Wolf, 2023; de Groot et al., 2021; Caravello et al., 2024). This framework does not care whether the impulse responses come from a DSGE model, a semi-structural model, or an econometric estimate. This creates a natural opportunity for common tooling that can perform this analysis given a set of models or impulse responses.

`ezdyn` provides such common tooling. Given a model – loaded either from a Dynare export or supplied as a generic table of impulse responses – and a small metadata file describing its variables, `ezdyn` provides simple command-line and interactive interfaces for calculating and plotting:[1]

- **Impulse responses**, including responses to a simple shock and responses to a targeted path for one or more variables (Section 3.2);
- **Historical shock decompositions** (Section 3.3);
- **Alternative policy paths** (Section 3.4); and
- **Optimal policy strategies** under a quadratic loss function, with commitment, discretion, and timeless-commitment variants (Section 3.5).

The remainder of this paper is organised as follows. Section 2 shows the dashboard first, since it is the simplest way to see what `ezdyn` can do. Section 3 walks through using `ezdyn` on the command line. Section 4 and Appendix B will discuss the sufficient-statistics theory behind policy counterfactuals and optimal policy.

See Appendix A.3 for start-to-finish instructions on getting started with ezdyn.

[1] A companion MATLAB toolkit, `ezDynare`, runs Dynare programmatically over arbitrary combinations of data vintages, sample ranges, and model variants, and exports the resulting model objects for use by `ezdyn` (see Appendix A).

2 The `ezdyn` dashboard

Most of what `ezdyn` can do can be accessed through a web dashboard. The dashboard is organised into four tabs.

2.1 Optimal policy

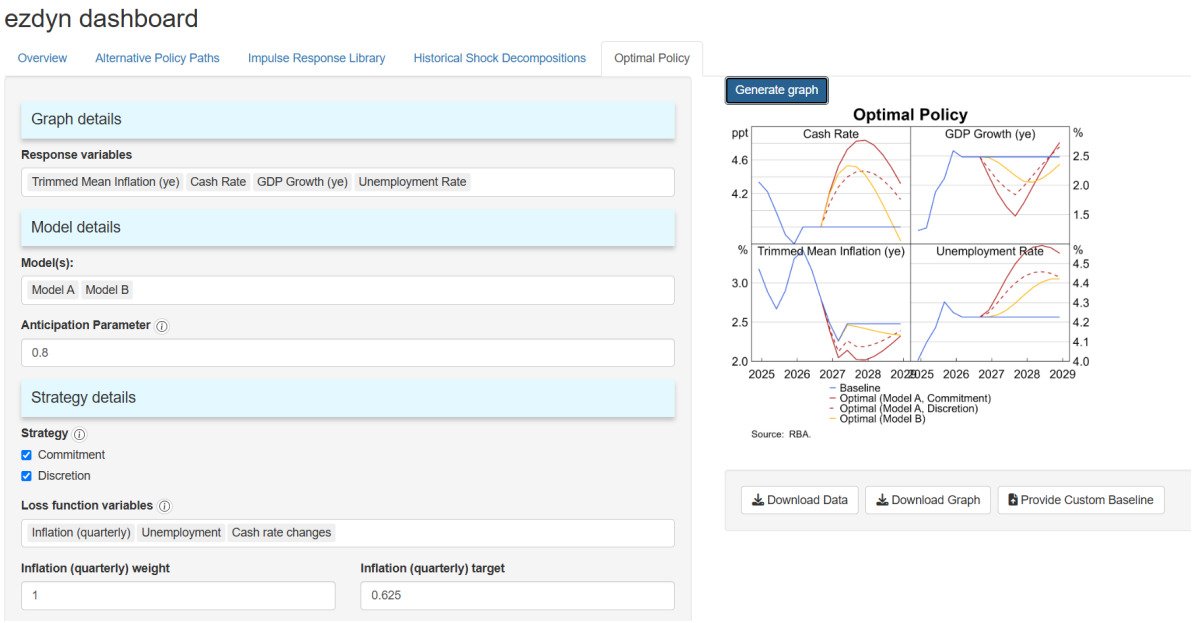


Figure 1: The Optimal Policy tab. The baseline paths plotted are dummy data, making the results of the exercise meaningless and illustrative.

The optimal policy tab allows users to complete sophisticated optimal monetary policy analysis across a suite of models interactively. The user selects a baseline scenario, a suite of models, commitment and/or discretion, and a loss function, and can see the optimal policy paths and implications for other macroeconomic variables.

In the 'advanced options', users can even specify constraints on the optimal policy problem, including introducing an effective lower bound constraint, requiring inflation to return to a target by a specified date, or imposing constraints on interest rate movements.

2.2 Alternative policy path counterfactuals

On the alternative policy paths tab, a user can draw an alternative policy path for a given economic scenario, and see what each model implies for the resulting macroeconomic variables.



Figure 2: The Alternative policy paths tab, showing the effects of a counterfactual nominal interest rate path according to two economic models. The baseline paths plotted are dummy data.

2.3 Impulse response library

On the impulse response library, users can explore the dynamic responses of macroeconomic variables to various shocks across different models. The tab has three sections: (1) impulse responses to a set of pre-configured economic scenarios (2) impulse responses where the user can scale the shock to hit a given target (3) impulse responses to a unit shock.

2.4 Historical shock decompositions

Users can decompose historical macroeconomic fluctuations into contributions from various shocks according to each model. The user can create interactive plots decomposing the different models, different variables. Shocks can be viewed individually or under different pre-configured groupings (e.g. supply/demand/foreign shock groups).

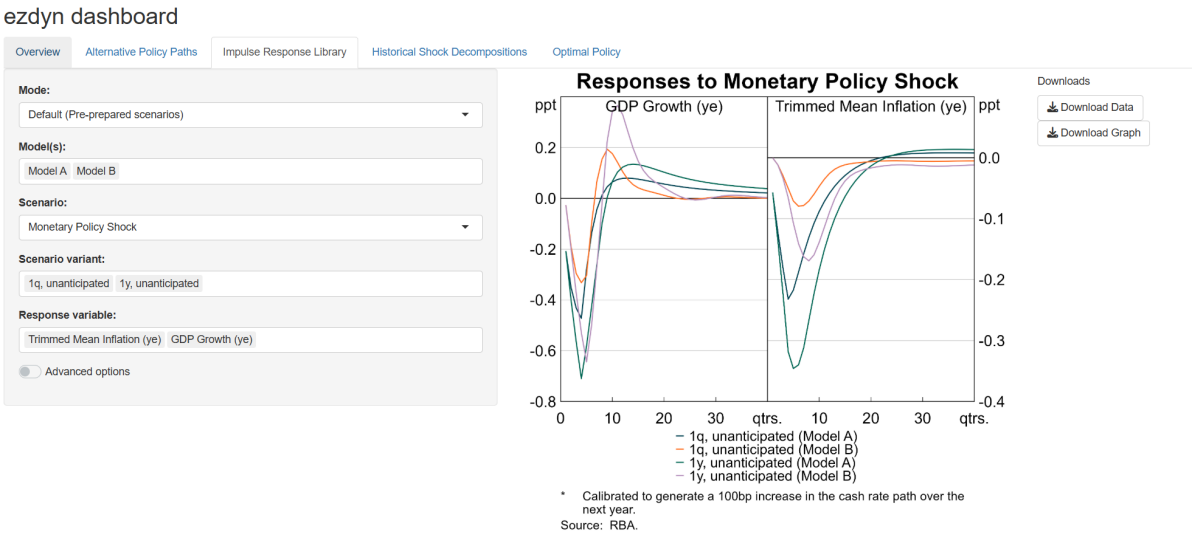


Figure 3: The IRF tab, showing a monetary policy shock under both unanticipated and anticipated variants.

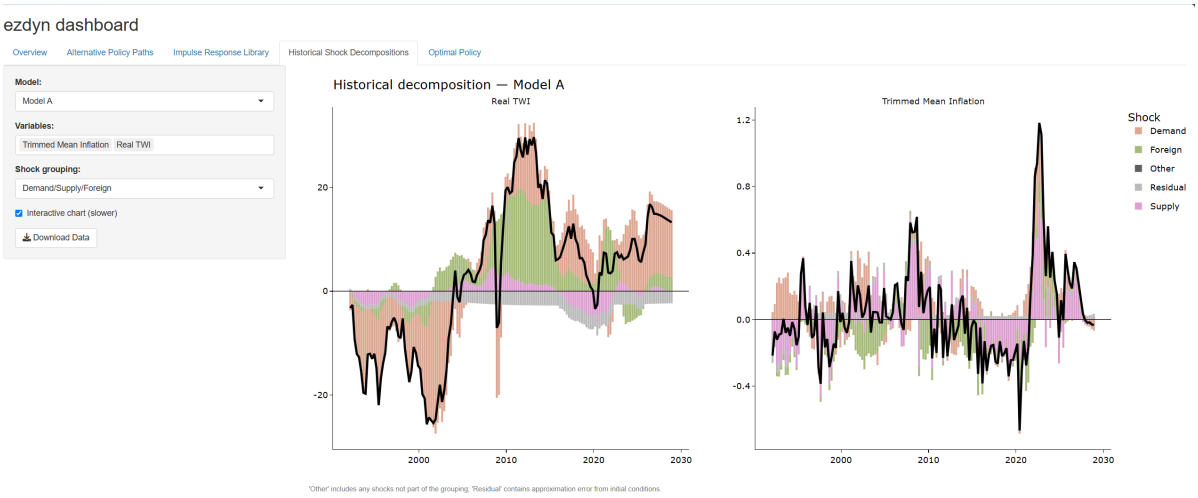


Figure 4: The Historical Shock Decompositions tab, showing the contributions of different shocks to historical macroeconomic fluctuations.

3 `ezdyn` on the command line

This section demonstrates each of the four core workflows. Every function below has additional arguments not shown here (for full documentation, use `?function_name` inside R).

3.1 Loading in a model

See Appendix A.3 for start-to-finish instructions on getting started with ezdyn.

`read_dynare()` reads in a model from a model object exported by `ezDynare`, and `custom_moo()` reads in a model from a list of impulse responses.[2] Both accept a metadata workbook giving each variable's display name, units, and an optional rescaling formula, so `ezdyn` can match the "same" variable across models by a shared `display_name`, regardless of each model's own naming or units.

```
library(ezdyn)

ModelA <- read_dynare("ModelA.json",
                      path_meta = "variable_metadata.xlsx",
                      model_name = "Model A")

ModelB <- custom_moo(ModelB_irf,
                     meta = "variable_metadata_ModelB.xlsx",
                     model_name = "ModelB")

models <- list(ModelA = ModelA, ModelB = ModelB)
```

3.2 Impulse responses

There are three main functions relevant to retrieving impulse responses for a set of models:

- `get_irfs()` returns the impulse response to one or more named shocks
- `get_irfs_target()` instead solves for the sequence of shocks required to hit a target path for one or more variables, then returns the resulting response of every endogenous variable – for example, "what shock sequence delivers a 100 then 125 basis-point cash-rate increase, and what does that imply for inflation and growth?"

```
target <- list("Cash Rate" = c(1, 1.25))
shock_timing <- list("Monetary policy shock" = 1:2)

irf <- get_irfs_target(models, horizon = 20,
                       target = target, shock_timing = shock_timing,
                       shock_nickname = "Cash-rate hike")
```

[2] Internally, both represent the model as an `M_`/`oo_` pair – following Dynare's convention, informally a "MOO" object – so that every `ezdyn` function works unchanged regardless of the underlying model.

```
plot_pretty(irf, display_names = c("Cash Rate", "Inflation", "GDP Growth"))$graph
```

3.3 Historical shock decomposition

`ez_hd()` converts a Dynare historical shock decomposition (or a supplied decomposition, for non-Dynare models) into a tidy, labelled table. `plot_pretty_hd()` plots it, optionally aggregating shocks into named groups (as preconfigured in model metadata) via `shock_group`:

```
hd <- ez_hd(models$ModelA$M_, models$ModelA$oo_)

plot_pretty_hd(hd, display_names = c("Inflation", "GDP Growth"),
               shock_group = "Demand/Supply/Foreign")$graph
```

3.4 Alternative policy paths

Alternative-path analysis starts from a common baseline forecast – the same canonical `baseline` built once in Section 2 via `import_baseline()` – and one or more candidate policy-rate paths. `get_alt_paths()` solves for the shocks needed to hit each candidate path in every model in `models` and returns a single combined, labelled result:

```
alt_paths <- get_alt_paths(
  alt_paths = readxl::read_excel("alternative_paths.xlsx"),
  models = models,
  baseline = baseline,
  forecast_start = as.Date("2026-09-01"),
  forecast_end = as.Date("2028-06-01"),
  data_start = as.Date("2020-03-01")
)

plot_pretty(alt_paths,
            display_names = c("Cash Rate", "Inflation", "GDP Growth"))$graph
```

3.5 Optimal policy

`get_policy_scenario()` implements the sufficient-statistics approach to optimal policy described in Section 4.1. It takes a `policy_strategy` describing a quadratic loss function – the loss variables, their relative weights, the discount factor, and the loss and instrument horizons – together with a baseline and a model. As with alternative paths, variables in `policy_strategy` may be given as display names which are consistent across models. The `commit` field selects between full commitment (`"commit"`), discretion (`"disc"`), and a timeless-commitment variant (`"timeless"`) that treats the strategy as though it had been in place since some earlier date.

```
strategy <- list(
  output_name = "Optimal (ModelA, commitment)",
  output_variables = c("Cash Rate", "Inflation", "Unemployment"),
```

```
  loss_variables = c("Inflation", "Unemployment", "Cash Rate"),
  loss_weights = c(1, 1, 0.5),
  discount_factor = 0.99,
  T_loss = 20,
  T_instrument = 20,
  commit = "commit"
)

optimal <- get_policy_scenario(strategy, baseline = baseline, model = models$ModelA
    ,
                               forecast_start = "2026-09-01",
                               forecast_end = "2031-06-01")

plot_pretty(optimal,
            display_names = c("Cash Rate", "Inflation", "Unemployment"))$graph
```

Under discretion, the loss and instrument horizons must coincide, since a purely forward-looking, re-optimising policymaker cannot condition on a loss horizon that extends beyond the horizon over which it controls the instrument.

An optional `constraints` entry in `policy_strategy` imposes a lower or upper bound on any variable over a chosen set of periods e.g. (an effective lower bound on the cash rate). Under timeless commitment, a constraint can additionally be marked `historical = TRUE`, which re-imposes it at every historical forecast vintage (not just the live one), so that a past effective-lower-bound episode, say, continues to generate 'make-up' policy today.

See Section 4.1 and Appendix B (forthcoming) for further background on these optimal policy concepts and their solution algorithms.

3.6 Parameter, variable, and shock tables

Finally, `get_param_table()` turns a model's calibrated and estimated parameter values into a table, using parameter labels and grouping from the metadata workbook's optional `parameters` sheet.[3]

```
get_param_table(models$ModelA$M_, dp = 3, as_of = "August 2026")
```

`get_variable_table()` and `get_shock_table()` are analogous functions for a model's variables and shocks, listing the display names/descriptions defined by the model's metadata workbook with the same `mode`/`show_codes` arguments:

```
get_variable_table(models$ModelA$M_, mode = "dt")
get_shock_table(models$ModelA$M_, mode = "dt")
```

[3]The `mode` argument returns static table (`"katex"`, the default, suited to reports) or an interactive, sortable/searchable table (`"dt"`, suited to a live dashboard) `show_codes = TRUE` adds the model's own raw parameter code alongside the rendered symbol.

4 Theoretical background

4.1 Sufficient statistics for policy counterfactuals and optimal policy

`ezdyn`'s policy-counterfactual workflows (Section 3.4) and optimal-policy workflows (Section 3.5) both rest on a "sufficient-statistics" relationship from the literature on policy counterfactuals and optimal policy (Barnichon and Mesters, 2023; McKay and Wolf, 2023; de Groot et al., 2021; Caravello et al., 2024): given only a baseline forecast, a matrix of policy impulse responses, and – for optimal policy – a loss function over target variables, the relevant counterfactual or optimal policy path (under commitment, discretion, or timeless commitment) can be computed directly in sequence space, without re-solving the underlying model.

Forthcoming.

References

Régis Barnichon and Geert Mesters. A sufficient statistics approach for macro policy. *American Economic Review*, 113(11):2809–2845, 2023. doi: 10.1257/aer.20220581.

Tomás E. Caravello, Alisdair McKay, and Christian Wolf. Evaluating monetary policy counterfactuals: (when) do we need structural models? NBER Working Paper 32988, National Bureau of Economic Research, 2024.

Oliver de Groot, Falk Mazelis, Roberto Motto, and Annukka Ristiniemi. A toolkit for computing Constrained Optimal Policy Projections (COPPs). ECB Working Paper 2555, European Central Bank, 2021.

Alisdair McKay and Christian Wolf. What can time-series regressions tell us about policy counterfactuals? *Econometrica*, 91(5):1695–1725, 2023. doi: 10.3982/ECTA21045.

A Getting started guide — Installation and setup

This appendix gives the minimum steps needed to load a new model into `ezdyn` and, optionally, to expose it through the dashboard. It may be split out into a separate "Getting Started" document as the package's public documentation matures.

A.1 Installing the package

Install R and RStudio (guide).

Once you have cloned the `ezdyn` repository, it can be installed using the R command:

```
devtools::install(".")
```

A.2 Setting up a model file

To run ezdyn, you need a model (either from dynare or a spreadsheet of IRFs) and a metadata spreadsheet containing metadata about the model's variables/shocks/parameters.

In this section, we detail how you can export a model from dynare, or setup a compatible IRF spreadsheet.

A.2.1 Exporting a model from MATLAB

The Dynare export consumed by `read_dynare()` is a JSON file containing a model's `M_`/`oo_` objects. The simplest way to produce one is to run Dynare as usual and then call `export_model_obj_json()` on the resulting objects:

```
dynare main.mod;
export_model_obj_json(M_, oo_, "model-objects/ModelA");
```

This writes `ModelA.json` straight into R's native JSON representation of `M_`/`oo_` (rather than Dynare's own `.mat` format, which R's `R.matlab` package cannot always read reliably), ready for `read_dynare()`.

For more involved workflows, `ezDynare` is a companion MATLAB toolkit that runs Dynare programmatically and calls `export_model_obj_json()` for you. See Section A.2.3 for an example usage.

A.2.2 Exporting a model into a IRF spreadsheet

Export all of your model's IRFs into a long-format data frame with columns `t`, `resp_var`, `shock`, and `value`.

A.2.3 Advanced: Using ezDynare to export models from MATLAB

Its design goals are: (i) Dynare code can be run with arbitrary variations without editing, copying, or pasting `.mod` files; (ii) the `.mod` file itself remains directly runnable outside `ezDynare`; and (iii) the user should not need to manually determine spreadsheet ranges for input data. A minimal call looks like:

```
input_data = 'Forecasts_Feb26'; % Dynare input data file
custom_args = struct(); % empty struct -> default macro arguments
ezDynare("main.mod 'nostrict' 'nograph'", input_data, custom_args, "Forecasts_Feb26
    ")
```

This determines the relevant spreadsheet range for `Forecasts_Feb26`, runs the model in Dynare, and saves the resulting `M_/oo_` objects (in a form `read_dynare()` can consume) to the model-objects folder. `ezDynare` also accepts a struct of custom macroprocessor arguments, which makes it straightforward to loop over combinations of data vintages, sample end dates, and model variants without duplicating `.mod` files.

A.3 Setting up a model metadata workbook

A model in `ezdyn` also requires a metadata workbook. This is used to translate the model's internal variables, shocks, and parameters into a user-friendly, cross-model consistent language.

Get started by downloading the sample metadata spreadsheet from this link https://github.com/RBA-ER-Replication/ezdyn/blob/main/tests/testthat/fixtures/dynare/test_metadata_dynare.xlsx and modifying it for your model.

The metadata workbook should include a sheet for 'variables' and (optionally) also a 'shocks' sheet, and a parameters sheet.

A.3.1 The variables sheet

At minimum this workbook needs a sheet with one row per variable and the columns:

- `dynare_name` – the model's own code for the variable;
- `display_name` – the name shown in plots and tables, and the key used to match the "same" variable across models;
- `units` and `unit_symbol` – the variable's units and the symbol used on plot axes; and
- `scale_formula` – an optional formula used to rescale the variable's raw model units into displayed units.

Optional columns include:

- a `default` column can flag which variables are shown by default;
- an `alt_paths` column can flag which variables are offered in the alternative-paths module;

- `units` and `unit_symbol` – the variable's units and the symbol used on plot axes; and
- `scale_formula` – an optional formula used to rescale the variable's raw model units into displayed units.

A.3.2 Shocks sheet

It is also recommended that you add a `shocks` sheet. It should include at a minimum:

- `shock` – the shock's code in the model;
- `description` – the name of the shock exposed to the user interface (e.g. Monetary Policy Shock).

Additional columns can be added as shock groupings. For example, you might add a column called 'Supply/Demand/Foreign', for which each shock would be categorized as either Supply, Demand, or Foreign. This is used for the historical shock decompositions functionality. You can add as many shock groupings as you like.

A.3.3 Parameters sheet

A separate sheet named `parameters`, with columns including `dynare_name` and a parameter grouping such as `Type`, is used by `get_param_table()` to build labelled parameter tables.

A.4 Loading in your model

A.4.1 From Dynare

```
ModelA <- read_dynare("ModelA.json",
                      path_meta = "variable_metadata.xlsx",
                      model_name = "Model A")
```

A.4.2 From an IRF spreadsheet

```
ModelB <- custom_moo(ModelB_irf,
                      model_name = "ModelB",
                      meta = "variable_metadata_ModelB.xlsx",
                      subset_vars = TRUE)
```

You are now free to use ezdyn, either on the command line (as discussed in the body of the manual) or through a dashboard interface. Setup for the dashboard interface is described in the following section.

A.5 Setting up an ezdyn dashboard

A dashboard is built from an explicit, validated configuration rather than global state: `configure_dashboard()` validates a named list of models, a forecast timeline, and one or more baselines – each built

once via `import_baseline()`, which translates a baseline forecast into the shared display-name space used across all configured models – and `ezdyn_dashboard()` turns the validated configuration into a ready-to-run dashboard application:

```
baseline <- import_baseline("baseline.xlsx", source_model = models$ModelA, models =
    models)

config <- configure_dashboard(
  name = "ezdyn Dashboard",
  models = models,
  timeline = list(data_start = as.Date("2010-03-01"),
                  data_end = as.Date("2026-06-01"),
                  forecast_start = as.Date("2026-09-01"),
                  forecast_end = as.Date("2029-06-01")),
  baselines = list("Staff Forecast" = baseline),
  default_baseline = "Staff Forecast"
)

shiny::runApp(ezdyn_dashboard(config))
```

This is launched with `shiny::runApp()` or deployed through the user's usual publishing workflow. A project-specific "composition root" – for example, a `dingo_dashboard()` function – typically wraps this configuration step so that end users only need to call one function to obtain a ready-to-run app.

B Algorithm for solving for optimal policy

Forthcoming.